So first, in our Gemfile, uncomment the Redis gem in the Gemfile like so:

```
gem 'redis', '~> 4.0'
```

Next, stop our Rails server:

```
$ docker-compose stop web
```

and rebuild our custom Rails image:

```
$ docker-compose build web
```

Among other things, this runs bundle install, which installs the Redis gem:

```
Building web
Step 1/8 : FROM ruby:2.6
«...»
Step 6/8 : RUN bundle install
«...»
Installing redis 4.1.0
«...»
Bundle complete! 16 Gemfile dependencies, 69 gems now installed.
Bundled gems are installed into `/usr/local/bundle`
«...»
Removing intermediate container 3831c10d2cb5
 ---> 1ca01125bd35
Step 7/8 : COPY . /usr/src/app/
 ---> 852dc1f2b419
Step 8/8 : CMD ["bin/rails", "s", "-b", "0.0.0.0"]
 ---> Running in 280c7e2eb556
Removing intermediate container 280c7e2eb556
 ---> d9b3e5325308
Successfully built d9b3e5325308
Successfully tagged myapp_web:latest
```

It's good to get into the habit of rebuilding our image to perform bundle install for us, having updated our Gemfile. That said, we'll learn about a more advanced approach to gem management on page 113 that, as well as being much faster, allows us to stick with our familiar bundle install workflow.

Let's start up our newly built Rails server again:

```
$ docker-compose up -d web
```

Updating Our Rails App to Use Redis

Next, we're going to actually use Redis from our Rails app. As we said before, we just want a basic demonstration that we can connect to the Redis server and store and retrieve values. So let's start by generating a welcome controller in our Rails app with a single index action: